

CHAPTER THREE

RESEARCH METHODOLOGY AND PROPOSED SYSTEM

3.1 Introduction

This chapter presents the research methodology adopted for the design and development of the proposed **Memo Management System** for **Koforidua Technical University (KTU)**. It explains the software development methodology used throughout the project, the reasons for selecting the methodology, and the activities performed during each stage of the software development life cycle.

The chapter further presents the proposed Memo Management System, describing its architecture, major functional components, system models, features, development tools, and implementation environment. Unified Modeling Language (UML) diagrams including the context diagram, sequence diagram, class diagram, and use case diagram are used to illustrate the interaction between users and the proposed system.

Finally, the chapter discusses the strengths and possible limitations of the proposed system and explains how the system addresses the communication challenges identified in Chapter One.

3.2 Software Development Methodology

3.2.1 Introduction to Agile Methodology

The **Agile Software Development Methodology** was adopted for the development of the proposed Memo Management System. Agile is an iterative and incremental software development methodology that emphasizes collaboration, flexibility, continuous improvement, and rapid delivery of functional software. Unlike traditional software development methodologies, Agile encourages regular interaction between developers and stakeholders throughout the development process.

Agile divides software development into short development cycles known as **sprints**, where each sprint focuses on implementing specific system features. At the end of every sprint, a working version of the software is produced, tested, evaluated, and improved based on feedback received from users or stakeholders.

The methodology promotes continuous testing, early identification of problems, and faster adaptation to changing user requirements. This makes Agile highly suitable for information systems whose requirements may evolve during development.

For the proposed Memo Management System, Agile enabled the incremental development of major modules such as user authentication, memo creation, memo approval, document tracking, notification management, user administration, and report generation.

3.2.2 Justification for Choosing Agile Methodology

The Agile methodology was selected because it provides flexibility throughout the software development process while ensuring continuous delivery of working software components. The proposed Memo Management System required regular improvements as new functional requirements emerged during system analysis and design. Agile made it possible to accommodate these changes without restarting the entire development process.

The following reasons justify the selection of Agile for this project.

1. Flexibility

The requirements of the Memo Management System evolved throughout the project. Agile allowed new features such as memo tracking, electronic approval workflows, notification services, and role-based access control to be incorporated during development without significantly affecting project progress.

2. Incremental Development

The project was developed module by module rather than as one large application. Each sprint produced a functional component of the system including:

1. User Registration Module
2. User Authentication Module
3. Memo Creation Module
4. Memo Approval Module
5. Memo Distribution Module
6. Memo Tracking Module
7. Notification Module

This incremental approach simplified development and made system testing more manageable.

3. Continuous Testing

Agile promotes testing throughout software development rather than waiting until coding has been completed. Every module developed for the Memo Management System was tested immediately after implementation, allowing defects to be identified and corrected early.

4. Improved Stakeholder Participation

The methodology encouraged regular consultation with prospective users including university administrators, departmental heads, registry staff, and other administrative personnel. Their feedback helped improve system usability and ensured that the final solution addressed the communication needs of Koforidua Technical University.

5. Faster Delivery of Functional Modules

Instead of waiting until the entire application was completed, Agile enabled functional modules to become operational immediately after each sprint. This approach improved productivity and allowed progressive evaluation of system performance.

6. Reduced Project Risk

By dividing the project into smaller development cycles, technical and design risks were minimized. Problems discovered during one sprint were resolved before the next sprint commenced, thereby improving software quality and reducing development costs.

3.2.3 Agile Software Development Process

The Agile software development process adopted for this project consisted of the following phases:

1. Requirement Gathering and Planning
2. System Analysis
3. System Design
4. Development and Implementation
5. Testing and Debugging
6. Deployment
7. Maintenance and Continuous Improvement

Each phase contributed significantly to the successful development of the proposed Memo Management System. The phases were executed iteratively, allowing improvements to be made whenever new requirements or feedback were received from stakeholders.

3.3 Activities Performed in Each Agile Phase

3.3.1 Requirement Gathering and Planning

The Requirement Gathering and Planning phase was the first stage in the development of the proposed Memo Management System. This phase focused on identifying the communication challenges experienced at Koforidua Technical University and determining the functional and non-functional requirements needed to address those challenges.

Information was gathered through observations of the existing memo handling process, discussions with administrative staff, review of institutional communication procedures, and literature reviewed in Chapter Two. The information collected helped define the scope, objectives, and overall functionality of the proposed system.

The following activities were performed during this phase:

Functional Requirements

The following functional requirements were identified:

1. User registration and authentication.
2. User login and logout.
3. Memo creation and editing.
4. Memo submission for approval.
5. Memo approval and rejection.
6. Memo forwarding to departments or users.
7. Memo tracking and status monitoring.
8. Memo search and retrieval.
9. Notification management.
10. Secure memo archiving.

Non-Functional Requirements

The following non-functional requirements were identified:

1. System security.
2. Reliability.
3. Performance.
4. Availability.

5. Scalability.
6. Maintainability.
7. Accessibility.

3.3.2 System Analysis

The System Analysis phase involved studying the existing memo handling process to understand its strengths and weaknesses. The objective was to identify the specific problems that needed to be addressed by the proposed Memo Management System.

The analysis revealed several challenges associated with the manual memo management process, including:

1. Delays in memo delivery.
2. Difficulty tracking memo status.
3. Loss or misplacement of documents.
4. High paper consumption.
5. Slow retrieval of archived memos.
6. Lack of centralized record management.
7. Difficulty monitoring user actions.
8. Limited communication transparency.

The interactions between these users and the proposed system were analyzed to determine the required system functionalities.

3.3.3 System Design

The System Design phase translated the identified requirements into a detailed blueprint for system development. During this phase, both the system architecture and database structure were designed.

The following design activities were carried out:

1. Database design using MySQL.
2. User interface design.
3. User authentication workflow.

4. Memo routing workflow.
5. Memo approval workflow.
6. Notification workflow.
7. User role and permission structure.

The following UML models were also designed:

1. Context Diagram
2. Use Case Diagram
3. Sequence Diagram
4. Class Diagram

These models provide a graphical representation of the proposed Memo Management System and facilitate communication among developers and stakeholders.

3.3.4 Development and Implementation

This phase involved the actual coding and implementation of the proposed Memo Management System based on the approved design specifications. The system was developed using modern web technologies to ensure flexibility, maintainability, scalability, and high performance.

The following technologies were used:

1. React.js
2. Express.js
3. MySQL Database
4. JavaScript (ES6+)
5. WebStorm IDE

The following modules were implemented:

1. Login and Authentication Module.
2. User Profile Module.
3. Memo Creation Module.
4. Memo Approval Module.
5. Memo Distribution Module.
6. Memo Tracking Module.
7. Notification Module.
8. Memo Archive Module.
9. User Management Module.

Each module was developed independently before being integrated into the complete system.

3.3.5 Testing and Debugging

Testing was conducted throughout the development process to ensure that every module functioned correctly and met the specified functional and non-functional requirements.

The following testing techniques were employed:

Unit Testing

Each module was tested independently before integration into the complete application.

Examples include:

1. Login and authentication validation.
2. Memo creation and editing.
3. Memo approval workflow.
4. Notification generation.
5. User management operations.

Integration Testing

The interaction among system components was tested to ensure seamless communication between:

1. React.js Frontend
2. Express.js Backend
3. MySQL Database
4. RESTful APIs

System Testing

The complete Memo Management System was tested to verify that all modules worked together correctly.

The testing focused on:

1. Functional correctness
2. Performance
3. Security
4. Reliability
5. Data integrity
6. Usability

User Acceptance Testing (UAT)

Prospective users, including administrative staff, registry personnel, and departmental representatives, evaluated the system to ensure that it satisfied the operational requirements and provided an intuitive user experience.

The following tools were used during testing:

1. WebStorm IDE.
2. Browser Developer Tools.
3. MySQL Workbench.
4. Postman (for API testing).

3.3.6 Deployment

After successful testing, the system was deployed in a controlled environment for demonstration and evaluation.

Deployment activities included:

1. Installation and configuration of the MySQL Database Server.
2. Configuration of the Express.js backend server.
3. Creation and initialization of the MySQL database.
4. Importation of database tables and sample data.
5. Deployment of the React.js frontend application.
6. Configuration of RESTful API endpoints.
7. Administrator account setup.
8. Final system verification and testing after deployment.

The deployment process ensured that the Memo Management System was fully operational and ready for use by authorized university staff.

3.3.7 Maintenance and Continuous Improvement

The Maintenance phase focuses on ensuring the continued effectiveness and reliability of the Memo Management System after deployment. During this phase, identified errors are corrected, system performance is improved, and new functionalities are added based on user feedback.

The following maintenance activities are expected:

1. Bug fixing.
2. Database optimization.
3. Security updates.

4. Performance improvements.
5. User support.
6. System monitoring.
7. Backup and recovery.

3.4 Proposed Memo Management System

3.4.1 Introduction to the Proposed System

The proposed system is a **web-based Memo Management System** designed to improve internal communication and document management at **Koforidua Technical University (KTU)**. The system provides a centralized platform through which authorized users can create, approve, distribute, track, retrieve, and archive official memoranda electronically.

Unlike the existing paper-based memo system, which relies heavily on manual document handling, the proposed Memo Management System automates the entire memo lifecycle. This automation minimizes delays in memo distribution, improves document security, reduces paper consumption, and enhances communication efficiency across departments and administrative units.

The proposed system incorporates role-based access control to ensure that users only perform activities that correspond to their responsibilities. For example, administrators can manage users and generate reports, departmental heads can approve or reject memos, while staff members can create, submit, receive, and acknowledge memos assigned to them.

The system also maintains a centralized database that securely stores memo records, user information, approval history, and communication logs. This improves record management, simplifies memo retrieval, and promotes accountability through audit trails.

In addition, the system provides automated notifications to inform users about newly assigned memos, approval requests, rejected memos, and other communication events. Read receipts and status tracking further enable administrators to monitor memo delivery and acknowledgement in real time.

Overall, the proposed Memo Management System is intended to improve administrative efficiency, strengthen institutional communication, support transparency, and provide a secure digital environment for managing official memoranda within Koforidua Technical University.

3.5 Models of the r

3.5.1 Context Model

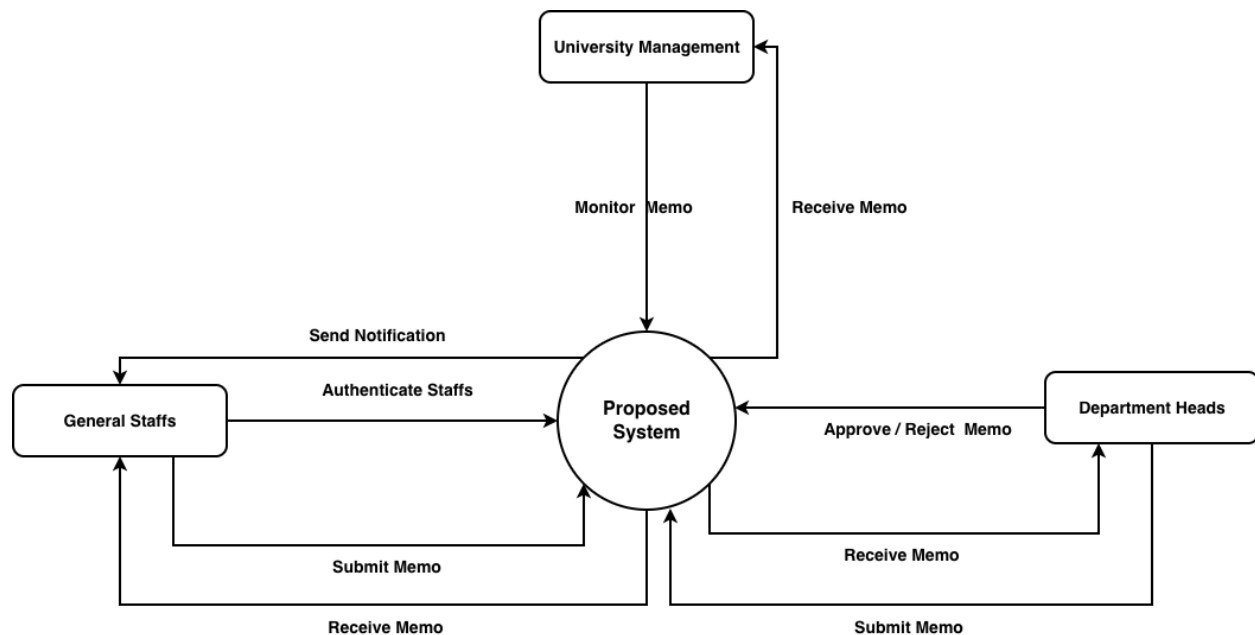


Figure 3.1: Context Diagram of the Proposed System

Description

The Context Diagram illustrates the interaction between the proposed Memo Management System and the external entities that use or support the system. It presents the system as a single central process and shows how information flows between the system and its users.

The primary external entities include the, **Department Heads**, **General Staff**, and the **University Management**.

Department Heads review submitted memoranda, approve or reject them, forward approved memos to intended recipients, and monitor departmental communications.

General Staff receive official memoranda, acknowledge receipt, read assigned documents, and search previously received communications where authorized.

University Management receives strategic memoranda, monitors institutional communication activities, and accesses reports generated by the system to support administrative decision-making.

The proposed Memo Management System processes requests received from these external entities and stores all relevant information within the centralized database. The outputs generated by the system include approved memoranda, notifications, acknowledgement reports, tracking information, communication history, and management reports.

The Context Diagram demonstrates that the Memo Management System serves as the central communication platform through which official memoranda are securely exchanged among all organizational units.

3.5.2 Interaction Model (Sequence Diagram)

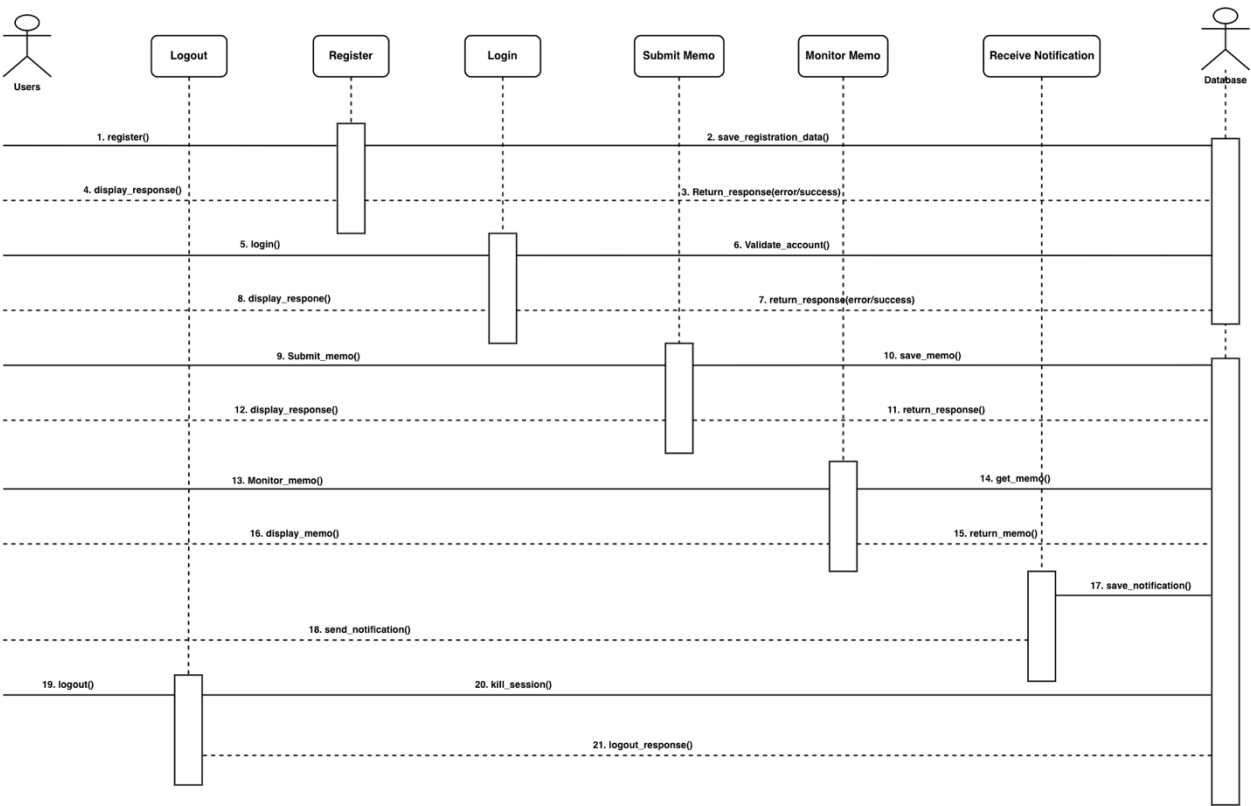


Figure 3.2: Sequence Diagram of the Proposed System

Description

The Sequence Diagram illustrates the chronological interactions that occur during the creation, approval, distribution, and acknowledgement of an official memorandum.

The process begins when an authorized staff member logs into the Memo Management System using a valid username and password. The authentication module validates the user's credentials before granting access to the system.

After successful authentication, the staff member selects the **Create Memo** option and enters the required information including the memo title, subject, recipient(s), priority level, department, and message content. Supporting documents may also be attached where necessary.

The completed memorandum is then submitted electronically to the appropriate Department Head or designated approving officer. The system records the submission and automatically generates a notification informing the approver that a new memo requires review.

The Department Head logs into the system, reviews the submitted memorandum, and decides whether to approve or reject it.

If the memo is approved, the system distributes it electronically to all intended recipients. Notifications are automatically generated to inform recipients that a new memorandum has been assigned to them.

Recipients access the memo through the system, read its contents, and acknowledge receipt electronically. The acknowledgement is recorded in the database, allowing administrators and senders to monitor the delivery status.

Throughout the process, every transaction is stored within the system database to maintain complete communication records and support future auditing.

The Sequence Diagram demonstrates how users, the application interface, the database, and the notification module interact to ensure secure and efficient electronic communication.

3.5.3 Structural Model (Class Diagram)

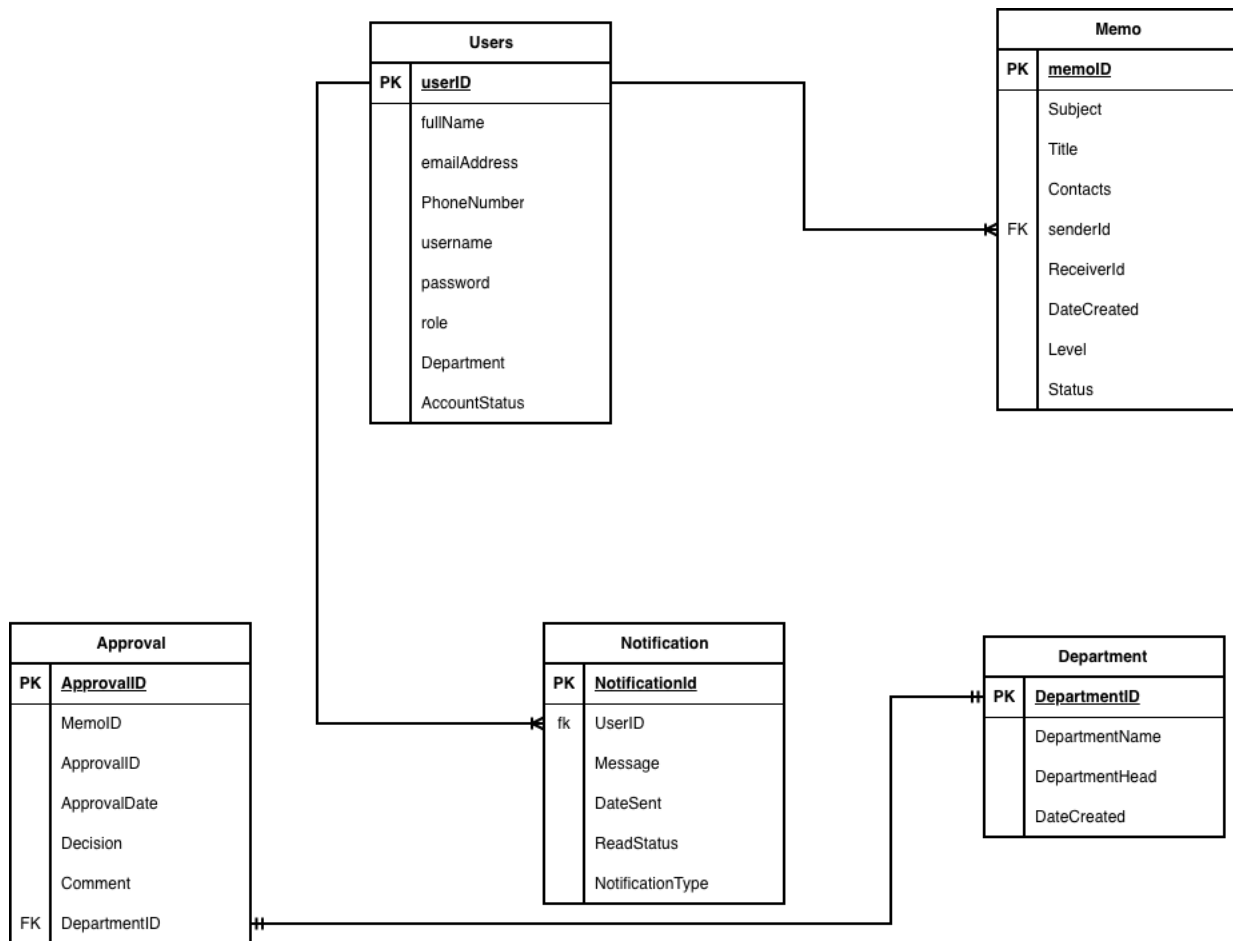


Figure 3.3: Class Diagram of the Proposed System

Description

The Class Diagram represents the structural design of the proposed Memo Management System by illustrating the major classes, their attributes, operations, and relationships.

The **User** class stores user information including cs, FullName, EmailAddress, PhoneNumber, Username, Password, Role, Department, and AccountStatus. It provides operations such as Login(), Logout(), UpdateProfile(), and ChangePassword().

The **Memo** class represents all memoranda created within the system. Its attributes include MemoID, Subject, Title, Content, SenderID, RecipientID, DateCreated, PriorityLevel, Status, and Attachment. Operations include CreateMemo(), EditMemo(), SubmitMemo(), ArchiveMemo(), and DeleteMemo().

The **Approval** class manages the memo approval process. It contains attributes such as ApprovalID, MemoID, ApproverID, ApprovalDate, Decision, and Comments. Its operations include ApproveMemo(), RejectMemo(), and ForwardMemo().

The **Notification** class manages communication alerts generated by the system. Attributes include NotificationID, UserID, Message, DateSent, ReadStatus, and NotificationType. Operations include SendNotification(), MarkAsRead(), and DeleteNotification().

The **Department** class stores departmental information including DepartmentID, DepartmentName, and DepartmentHead. It supports operations for managing departmental records and assigning users.

The **Report** class generates administrative reports relating to memo activities, user performance, pending approvals, communication statistics, and archived memoranda.

The relationships among these classes ensure efficient communication, secure information management, and effective administration of electronic memoranda.

3.5.4 Behavioral Model (Use Case Diagram)

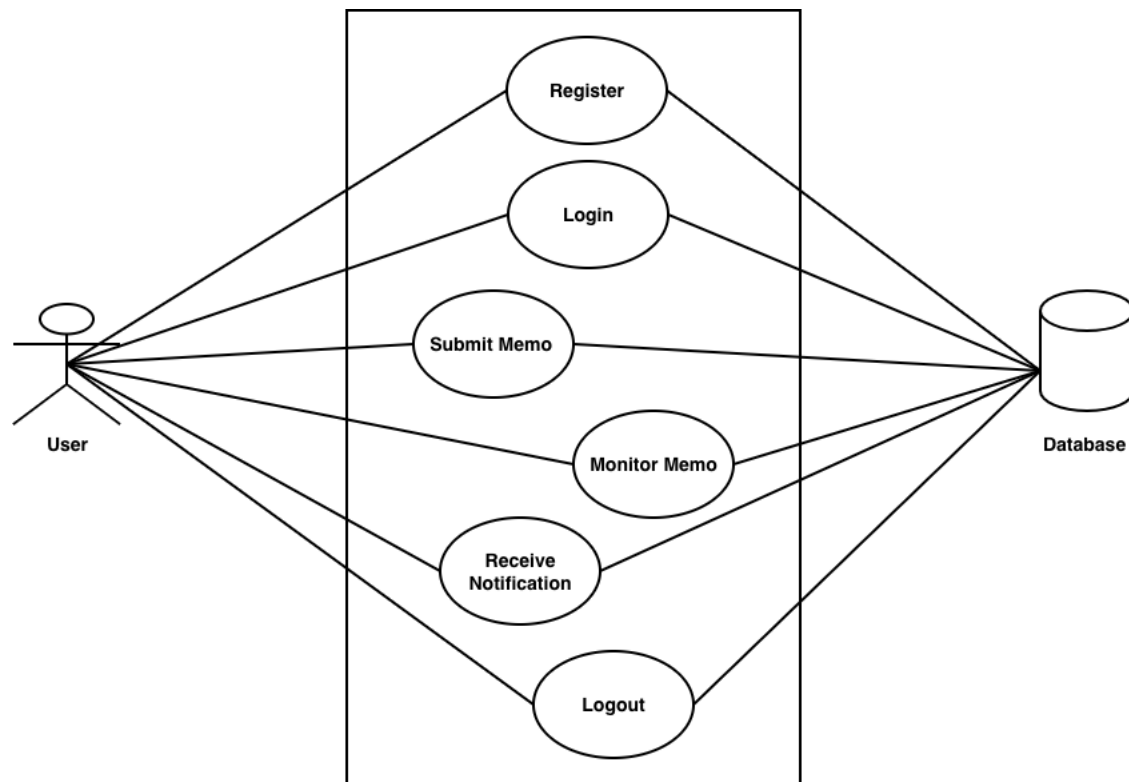


Figure 3.4: Use Case Diagram of the Proposed System

Description

The Use Case Diagram illustrates the interactions between the various users of the proposed Memo Management System and the services provided by the system. It identifies the primary actors and the major functions that each actor performs.

The **System Administrator** is responsible for managing the overall operation of the system. The administrator can create and manage user accounts, assign user roles and permissions, reset passwords, manage departments, generate reports, monitor system activities, and maintain the database. The administrator also oversees the security of the system by ensuring that only authorized users have access to sensitive information.

The **Department Head** is responsible for reviewing memoranda submitted by staff members. The Department Head can approve or reject memos, forward approved memoranda to recipients, monitor departmental communications, search archived memos, and generate departmental reports. These activities ensure that official communications are properly reviewed before distribution.

The **Administrative Staff** create new memoranda, edit drafts, submit memos for approval, track the progress of submitted memoranda, receive notifications, retrieve previous communications, and acknowledge received memos. Administrative staff also update their personal profiles and change their passwords when necessary.

General Staff receive official memoranda, read assigned communications, acknowledge receipt, search previously received memos, and manage their user profiles. These functions ensure effective communication between management and employees.

The **Database** supports all system operations by storing user information, memo records, approval history, notification records, and communication logs. Every activity performed within the system is recorded to ensure accountability, transparency, and efficient information retrieval.

Overall, the Use Case Diagram demonstrates how different categories of users interact with the Memo Management System to facilitate secure, efficient, and paperless institutional communication.

3.6 Features of the Proposed Memo Management System

The proposed Memo Management System incorporates several features designed to improve communication, document management, and administrative efficiency within Koforidua Technical University.

The major features include:

1. Secure user authentication and login.
2. Memo creation and editing.

3. Memo submission for approval.
4. Electronic memo approval and rejection.
5. Memo acknowledgement (Read Receipt).
6. Memo status tracking.
7. Automated notification system.
8. User profile management.
9. Memo attachment support.
10. Administrative dashboard.

These features collectively improve communication efficiency, reduce administrative workload, and promote effective management of official memoranda.

3.7 Development Tools and Development Environment

The proposed Memo Management System was developed using modern web technologies that provide reliability, scalability, security, and ease of maintenance. The selected technologies support the development of dynamic web applications capable of handling multiple concurrent users efficiently.

3.7.1 React.js

React.js was used as the primary front-end library for building the user interface of the Memo Management System. It enables the creation of reusable UI components and ensures a fast, responsive, and interactive user experience.

Advantages

1. Component-based architecture for reusable UI elements.
2. Fast rendering using virtual DOM.
3. Improved user experience with dynamic interfaces.
4. Strong community support and ecosystem.
5. Easy integration with backend APIs.
6. Supports modern single-page application (SPA) development.

3.7.2 Express.js

Express.js was used as the backend framework for building the server-side logic and RESTful APIs. It handles HTTP requests, processes business logic, and communicates with the MySQL database.

Advantages

1. Lightweight and fast backend framework.
2. Easy creation of RESTful APIs.
3. High flexibility and scalability.
4. Middleware support for request handling.
5. Strong integration with Node.js ecosystem.
6. Efficient handling of asynchronous operations.

3.7.3 JavaScript (ES6+)

JavaScript was used for client-side interactivity and communication with backend APIs. It enhances user experience by enabling dynamic page updates without reloading.

Advantages

1. Fast execution on the client side.
2. Supports asynchronous communication (AJAX/Fetch API).
3. Enables interactive web applications.
4. Reduces server load.
5. Works seamlessly with React and Express.

3.7.4 MySQL Database

MySQL was selected as the relational database management system for storing and managing system data such as users, memos, approvals, notifications, and logs.

Advantages

1. Reliable and secure data storage.
2. High performance for large datasets.
3. Supports multi-user environments.
4. Easy integration with Express.js via drivers (e.g., mysql2).
5. Strong data integrity and consistency.
6. Efficient backup and recovery mechanisms.

3.7.5 WebStorm IDE

WebStorm was used as the Integrated Development Environment (IDE) for developing both the frontend and backend of the system. It provides advanced tools for JavaScript development.

Advantages

1. Intelligent code completion and suggestions.
2. Built-in debugging tools.
3. Excellent support for React.js and Node.js.
4. Git version control integration.
5. Powerful refactoring tools.
6. Improves development productivity.

3.8 Review of the Good Features of the Proposed System

The proposed Memo Management System provides several advantages over the existing manual memo handling process at Koforidua Technical University. The system was designed to improve communication, document management, administrative efficiency, and institutional accountability through the application of modern information technology.

One of the major strengths of the proposed system is its **centralized electronic memo management** capability. All memoranda are created, approved, distributed, and archived within a single platform, eliminating the need for multiple paper files and reducing the risk of document loss. Centralized storage also enables users to retrieve previous memoranda quickly whenever required.

Another important feature is the **automated memo approval workflow**. Instead of physically moving documents from one office to another, memoranda are electronically routed to the appropriate approving officers. This significantly reduces delays in decision-making and improves administrative efficiency.

The system also provides **real-time notifications**. Whenever a memo is submitted, approved, rejected, or forwarded, the relevant users automatically receive notifications. This ensures that important communications are delivered promptly and minimizes delays caused by manual communication methods.

A significant advantage of the proposed system is its **memo tracking functionality**. Administrators and memo originators can monitor the status of every memorandum from creation to final acknowledgement. This tracking mechanism improves transparency and accountability by allowing users to determine whether a memo has been viewed, acknowledged, or is still pending.

The implementation of **role-based access control** enhances system security. Different categories of users—including administrators, department heads, registry staff, and general staff—have different permissions according to their responsibilities. This prevents unauthorized access to confidential institutional information.

The proposed system also improves **document security** through user authentication, password protection, and controlled access to electronic records. Sensitive memoranda remain protected from unauthorized users while maintaining data integrity.

Another important benefit is the **electronic archive**. All memoranda are stored in a structured database that supports efficient searching and retrieval using keywords, dates, departments,

subjects, or memo numbers. This eliminates the lengthy manual process of searching through paper files.

The system promotes **paperless communication**, reducing paper consumption, printing costs, storage requirements, and environmental impact. This supports Koforidua Technical University's commitment to sustainable administrative practices.

Furthermore, the system provides **administrative reporting capabilities**. Managers and administrators can generate reports on memo activities, pending approvals, user performance, communication history, and departmental correspondence. These reports assist management in monitoring institutional communication and making informed decisions.

The web-based architecture of the proposed Memo Management System enables users to access the system from different offices within the university network using standard web browsers. This improves collaboration among departments and facilitates efficient information sharing.

Overall, the proposed Memo Management System offers an efficient, secure, transparent, and reliable solution for managing institutional memoranda. The system significantly improves communication processes while reducing administrative workload and operational costs.

3.9 Review of the Bad Features (Limitations) of the Proposed System

Although the proposed Memo Management System offers numerous advantages, it also has certain limitations that should be considered during implementation and future system improvement.

One limitation is the system's **dependence on computer systems and network infrastructure**. Users must have access to computers and a reliable local area network or internet connection before they can create, approve, or retrieve memoranda. Network failures may temporarily interrupt communication activities.

Another limitation is the requirement for **digital literacy**. Some staff members who are accustomed to traditional paper-based communication may require training before they can effectively use the electronic memo management system.

The system also depends on **continuous electrical power** for its operation. Power interruptions may temporarily affect system availability if appropriate backup power solutions are not implemented.

Like all web-based applications, the proposed system is exposed to potential **cybersecurity threats** such as unauthorized access, malware attacks, phishing attempts, and data breaches. Regular software updates, strong authentication mechanisms, firewalls, and database security measures are therefore essential to protect institutional information.

Another limitation is the **cost of deployment and maintenance**. Although the software itself may utilize open-source technologies, organizations may still incur expenses relating to server hardware, networking equipment, system administration, staff training, and ongoing maintenance.

The system also requires **regular data backup and database maintenance** to ensure that institutional records are protected against accidental loss, hardware failures, or software errors.

Furthermore, successful implementation depends on effective **change management**. Staff members must be willing to transition from manual paper-based procedures to electronic communication practices. Resistance to organizational change may initially affect system adoption.

Despite these limitations, the benefits of the proposed Memo Management System significantly outweigh its challenges. Most of the identified limitations can be minimized through proper planning, adequate user training, reliable infrastructure, and continuous system maintenance.

3.10 Summary of the Chapter

This chapter presented the research methodology adopted for the development of the proposed Memo Management System for Koforidua Technical University. The Agile Software Development Methodology was selected because of its flexibility, iterative approach, and ability to accommodate continuous improvement throughout the software development process.

The chapter described the various phases of Agile development, including requirement gathering and planning, system analysis, system design, development and implementation, testing, deployment, and maintenance. The activities performed during each phase were explained in detail to demonstrate how the proposed system was developed systematically.

The chapter also introduced the proposed Memo Management System and described its architecture using Unified Modeling Language (UML) models, including the Context Diagram, Sequence Diagram, Class Diagram, and Use Case Diagram. These models illustrate the interaction between users and the system, the structure of the application, and the flow of information throughout the memo management process.

In addition, the chapter discussed the major features of the proposed system, the development tools and technologies used during implementation, and the strengths and limitations of the system. The proposed Memo Management System provides a secure, centralized, and efficient platform for creating, approving, distributing, tracking, retrieving, and archiving official memoranda. By automating manual communication processes, the system is expected to improve administrative efficiency, enhance transparency and accountability, reduce paper consumption, and support effective institutional communication within Koforidua Technical University.